MSA-DATB09-03 · NEURAL NETWORKS & DEEP LEARNING FOUNDATIONS

Session 6 / 8

Training better networks

Regularisation, normalisation, optimisers, and how to read a training run.

Guillaume Saint-Cirgue Albert School · MSc 1 October 2026

Guillaume Saint-Cirgue your instructor

Data Scientist and ML Engineer since 2016

2013 to 2016	Engineering degree, **ENSMM** (Besançon)
2016 to 2017	Data Scientist, **Rolls-Royce**
2017 to 2019	Lead ML Engineer, **GKN Aerospace**
2019	Launched **Machine Learnia** on YouTube: 200,000+ subscribers today
2020	Founded my company and my startup: **30+ AI projects** delivered since
2023	Head of Data, **Airbus**
Today	**ML-PRO** training program: 800 students

To go further

MY YOUTUBE CHANNEL

Machine Learnia

200,000+ subscribers. Machine learning and deep learning videos, including a neural network coded from scratch in NumPy.

youtube.com/@MachineLearnia

MY EBOOK

Apprendre le Machine Learning en une semaine

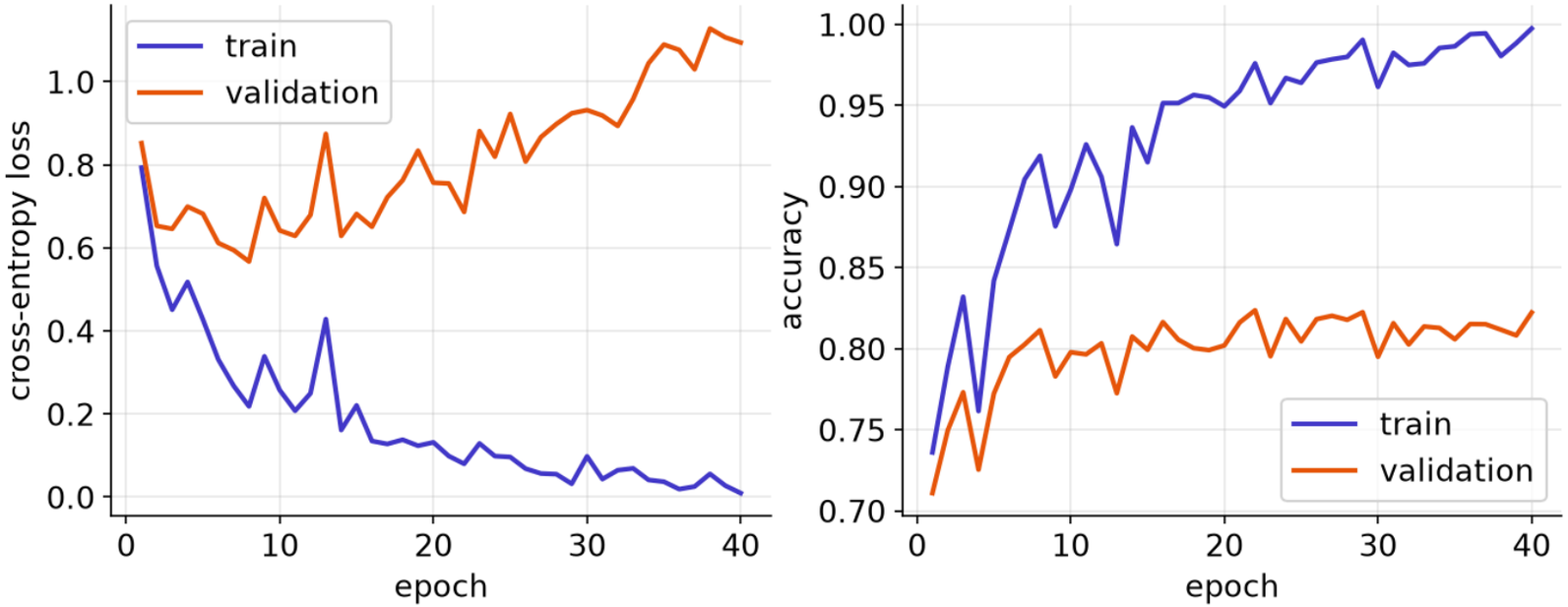
In English: Learn Machine Learning in One Week.

machinelearnia.com/apprendre-le-machine-learning-en-une-semaine

Both resources are in French. Click the cards in your copy of the course to open them.

Last time the network trained. Now it learns too well

A 784-512-512-10 ReLU network, trained on only **2 000** FashionMNIST images:



Training loss goes to almost 0; validation loss is best at epoch 8, then rises.

After 40 epochs: train accuracy 0.998, validation accuracy 0.822. The network has **memorised** the training set, and it becomes over-confident on validation images (the loss rises).

Overfitting: the gap between train and validation

- **Underfitting** (high bias): train error high. The model is too simple or not trained enough.
- **Overfitting** (high variance): train error low, validation error much higher. The model fits the noise of its training set.
- A big network on little data overfits: it has far more parameters than examples (here 670 000 parameters for 2 000 images).

Today's tool	Idea
Weight decay (L2)	keep the weights small
Dropout	do not rely on any single neuron
Early stopping	stop at the best validation epoch
Batch norm	keep each layer's inputs well scaled
Momentum, Adam	better steps than plain SGD

The first fix is always more data. When we cannot, we regularise.

L2 regularisation: penalise large weights

We add a term to the cost J (the mean loss over the dataset):

$$J_{\text{reg}}(W) = J(W) + \frac{\lambda}{2}\,\|W\|^2, \qquad \|W\|^2 = \sum_{l}\sum_{j,k}\left(W_{jk}^{[l]}\right)^2$$

- $\lambda \geq 0$ sets the strength (typical values 10^{-5} to 10^{-2}). Biases are usually not penalised.
- Why it helps: a network with small weights computes a smoother function, which cannot follow every noisy training point.

The gradient of the extra term, weight by weight:

$$\frac{\partial}{\partial w}\left(\frac{\lambda}{2}w^2\right) = \lambda\, w \quad \Rightarrow \quad \frac{\partial J_{\text{reg}}}{\partial w} = \frac{\partial J}{\partial w} + \lambda\, w$$

The update becomes "weight decay"

Gradient descent on J_{reg}, step by step:

$$w \leftarrow w - \eta\left(\frac{\partial J}{\partial w} + \lambda w\right) = \underbrace{(1 - \eta\lambda)}_{\text{slightly below } 1}\ w\ -\ \eta\,\frac{\partial J}{\partial w}$$

- At every step the weight is first multiplied by $1 - \eta\lambda$ (it "decays" towards 0), then the usual step is applied.
- A weight that the data do not need shrinks to 0; a useful weight is kept by its gradient.

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.1, weight_decay=1e-3)    # adds lambda * w to every gradient
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-2) # decoupled version, for Adam
```

Exercise 1 · One update with weight decay

A layer has two weights $w = (2, -1)$. The backward pass gives $\partial J / \partial w = (0.5, -0.5)$. We use $\eta = 0.1$.

Exercise 7 min pen and paper

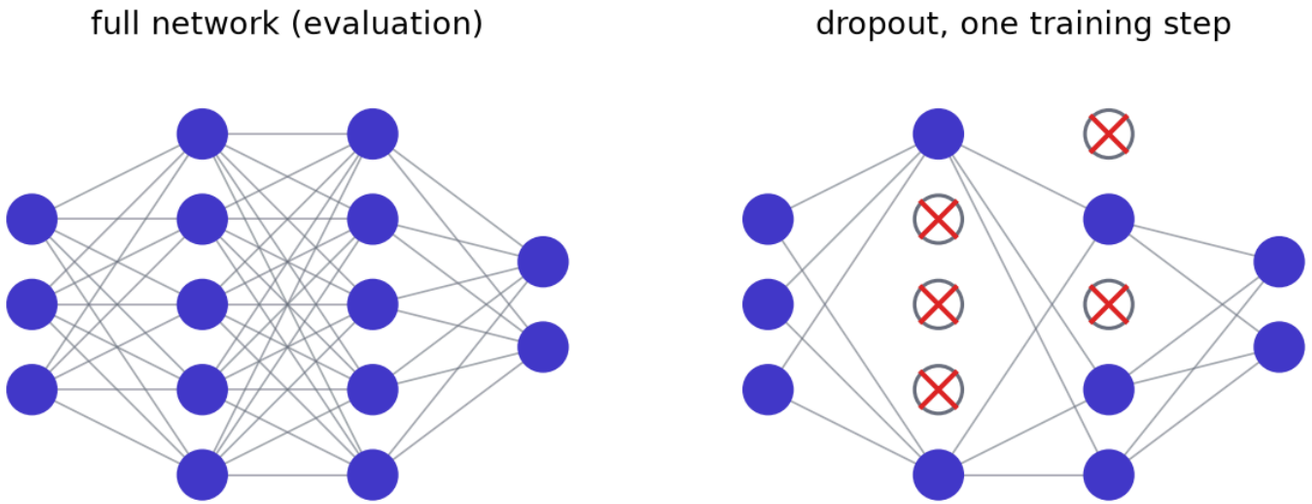
1. Compute the new weights without regularisation.
2. Compute the new weights with weight decay $\lambda = 0.5$. What is the decay factor $1 - \eta\lambda$?
3. Compute the penalty $\frac{\lambda}{2}\|w\|^2$ at the start.
4. If the data gradient were 0, after how many steps would a weight be halved?

Your notes / answers (saved in this browser)

Dropout: switch off random neurons during training

- At each training step, every hidden neuron is **dropped** (output set to 0) with probability p, independently.
- Each step trains a different thinned network.
- A neuron cannot rely on one specific other neuron: the network learns redundant features.
- At test time: no dropout, the full network.

Srivastava, Hinton et al., 2014. Typical p: 0.1 to 0.5.



Inverted dropout: why we divide by $1 - p$

For the activations a of a layer, draw a mask $m_i \in \{0, 1\}$ with $P(m_i = 1) = 1 - p$, then:

$$\tilde{a}_i = \frac{m_i \, a_i}{1 - p} \qquad (\text{training}), \qquad\qquad \tilde{a}_i = a_i \qquad (\text{evaluation})$$

Expected value during training:

$$\mathbb{E}[\tilde{a}_i] = \frac{a_i}{1 - p} \, \mathbb{E}[m_i] = \frac{a_i}{1 - p} \big(1 \cdot (1 - p) + 0 \cdot p\big) = a_i$$

- On average the next layer receives the same signal in training and at test time: nothing to change at test time.
- Without the $1/(1 - p)$, the next layer would see inputs about $1/(1 - p)$ times larger at test time than in training.

Dropout in PyTorch: train mode versus eval mode

```
import torch, torch.nn as nn
torch.manual_seed(0)
drop = nn.Dropout(p=0.5)      # p = probability to DROP a value
a = torch.ones(8)
drop.train()                  # training mode (what model.train() does)
print(drop(a))                # tensor([0., 0., 2., 0., 0., 0., 2., 2.])  kept values x 1/(1-p) = 2
drop.eval()                   # evaluation mode (what model.eval() does)
print(drop(a))                # tensor([1., 1., 1., 1., 1., 1., 1., 1.])  identity
```

The classic bug

Forgetting `model.eval()` before evaluating: predictions become random and accuracy drops.
Forgetting `model.train()` after: dropout is silently switched off for the rest of training.

Exercise 2 · Apply a dropout mask

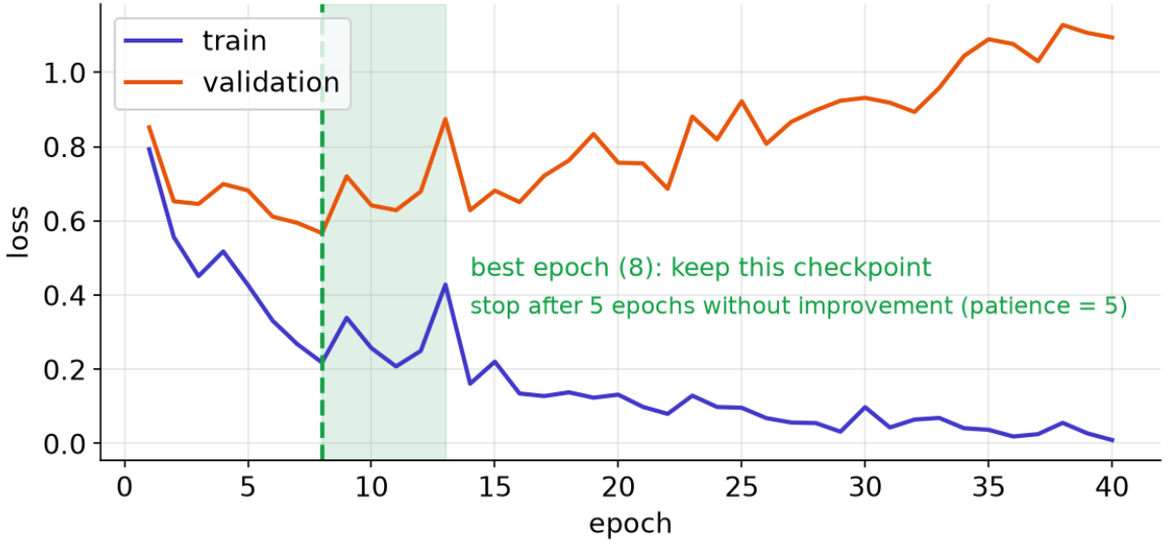
A hidden layer outputs $a = (2, 1, 4, 3)$. Dropout with $p = 0.5$; the random mask drawn for this step is $m = (1, 0, 1, 0)$.

Exercise 6 min pen and paper

1. Compute $\tilde{a}$ in training mode (inverted dropout).
2. Compare the sum of $\tilde{a}$ with the sum of a. Is that a problem?
3. What is the expected value of $\tilde{a}_1$ over all possible masks?
4. What does the layer output in evaluation mode?

Your notes / answers (saved in this browser)

Early stopping: keep the best validation model




```
best_loss, best_state, wait = float("inf"), None, 0
for epoch in range(max_epochs):
    train_one_epoch(model, train_loader, loss_fn, optimizer)
    val_loss, val_acc = evaluate(model, val_loader, loss_fn)
    if val_loss < best_loss:                                    # improvement: save a checkpoint
        best_loss, best_state, wait = val_loss, copy.deepcopy(model.state_dict()), 0
    else:
        wait += 1
        if wait ⩾ patience: break                               # no improvement for `patience` epochs
model.load_state_dict(best_state)                               # go back to the best epoch
```

Batch normalisation: the formulas

For one neuron, over a mini-batch of m pre-activations $z^{(1)}, \dots, z^{(m)}$:

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} z^{(i)}, \quad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}\left(z^{(i)} - \mu_B\right)^2, \quad \hat{z}^{(i)} = \frac{z^{(i)} - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad y^{(i)} = \gamma\,\hat{z}^{(i)} + \beta$$

- $\hat{z}$ has mean 0 and variance 1 over the batch; $\varepsilon \approx 10^{-5}$ avoids a division by 0.
- γ and β are **learned** (one pair per neuron): the network can choose another scale, even undo the normalisation.
- Placed between the linear layer and the activation: `Linear`, `BatchNorm1d`, `ReLU`.

Why it helps

Each layer receives well-scaled inputs whatever happens before it: training is faster, tolerates larger learning rates and is less sensitive to the initialisation (Ioffe and Szegedy, 2015).

Batch norm at test time: running statistics

- At test time we may predict one image alone: there is no batch to average over.
- During training, PyTorch keeps an exponential moving average of the batch statistics (`momentum = 0.1`):

$$\mu_{\text{run}} \leftarrow 0.9\,\mu_{\text{run}} + 0.1\,\mu_B, \qquad \sigma^2_{\text{run}} \leftarrow 0.9\,\sigma^2_{\text{run}} + 0.1\,\frac{m}{m-1}\sigma^2_B$$

```
bn = nn.BatchNorm1d(1)                  # gamma = 1, beta = 0, running mean 0, running var 1
z = torch.tensor([[2.], [4.], [6.], [8.]])   # a batch of 4 examples, 1 neuron
bn.train(); print(bn(z).flatten())      # tensor([-1.3416, -0.4472, 0.4472, 1.3416])
print(bn.running_mean, bn.running_var)  # tensor([0.5000]) tensor([1.5667])
bn.eval();  print(bn(z).flatten())      # tensor([1.1984, 2.7963, 4.3941, 5.9920])
```

Running mean $0.1 \times 5 = 0.5$, running variance $0.9 + 0.1 \times 20/3 = 1.567$. In `eval()` mode these running statistics are used (after one batch they are still far from the truth, hence different outputs): one more reason never to forget `model.eval()`.

Exercise 3 · Batch norm on a batch of 4 numbers

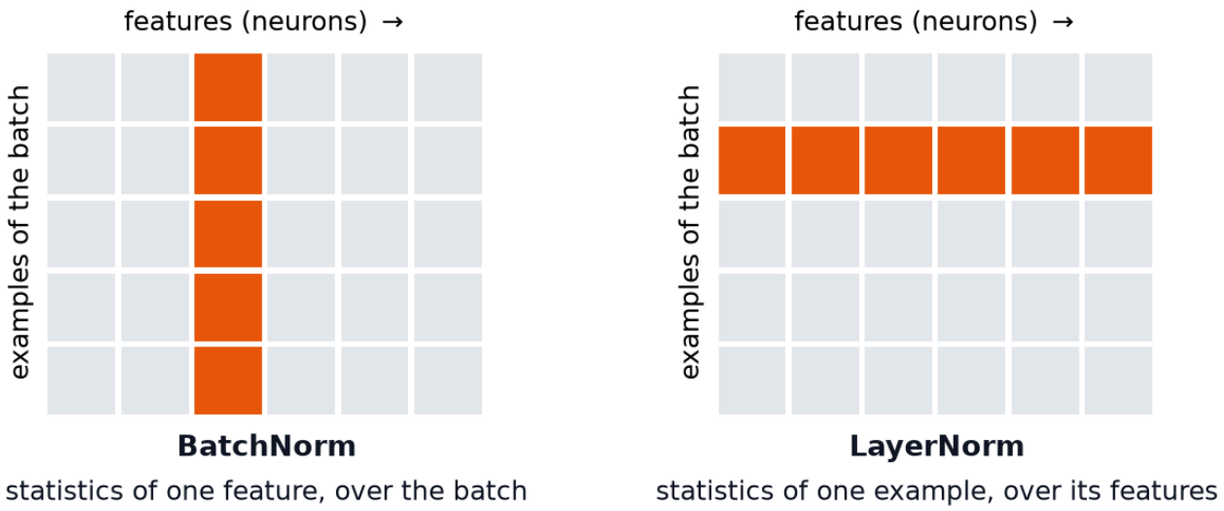
One neuron, a batch of 4 pre-activations $z = (1, 3, 3, 9)$. Learned parameters $\gamma = 2$, $\beta = 0.5$. Neglect ε.

Exercise 7 min pen and paper

1. Compute μ_B and σ_B^2.
2. Compute the normalised values $\hat{z}$. Check that their mean is 0.
3. Compute the outputs $y = \gamma \hat{z} + \beta$.
4. The running mean starts at 0 (momentum 0.1). What is it after this batch?

Your notes / answers (saved in this browser)

Layer normalisation: the other axis



BatchNorm: CNNs

Statistics of one neuron (one channel for images) over the batch. Needs reasonable batches (32 or more); different behaviour in train and eval mode. The standard in CNNs (ResNet).

LayerNorm: transformers

Statistics of one example over its own features. Independent of the batch size and of the other examples, identical in train and eval. Works for sequences of any length: the standard in transformers and RNNs.

Better steps: momentum

Plain (stochastic) gradient descent: $w \leftarrow w - \eta\, g$, with $g = \partial J / \partial w$ on a mini-batch.

- In a narrow valley it zigzags across the steep direction and crawls along the flat one.
- Mini-batch gradients are noisy.

Momentum: accumulate a velocity v (start $v = 0$):

$$v \leftarrow \beta\, v + g, \qquad w \leftarrow w - \eta\, v$$

- $\beta = 0.9$ typically: v is a running sum of past gradients, $v = g_t + \beta g_{t-1} + \beta^2 g_{t-2} + \dots$
- Components that change sign (zigzag) cancel out; components that keep the same sign add up.
- With a constant gradient, $v \to g/(1-\beta) = 10\, g$: up to 10 times bigger steps along a consistent direction.

This is PyTorch's convention (`SGD(..., momentum=0.9)`). Some textbooks write $v \leftarrow \beta v + (1-\beta) g$: same idea, η rescaled.

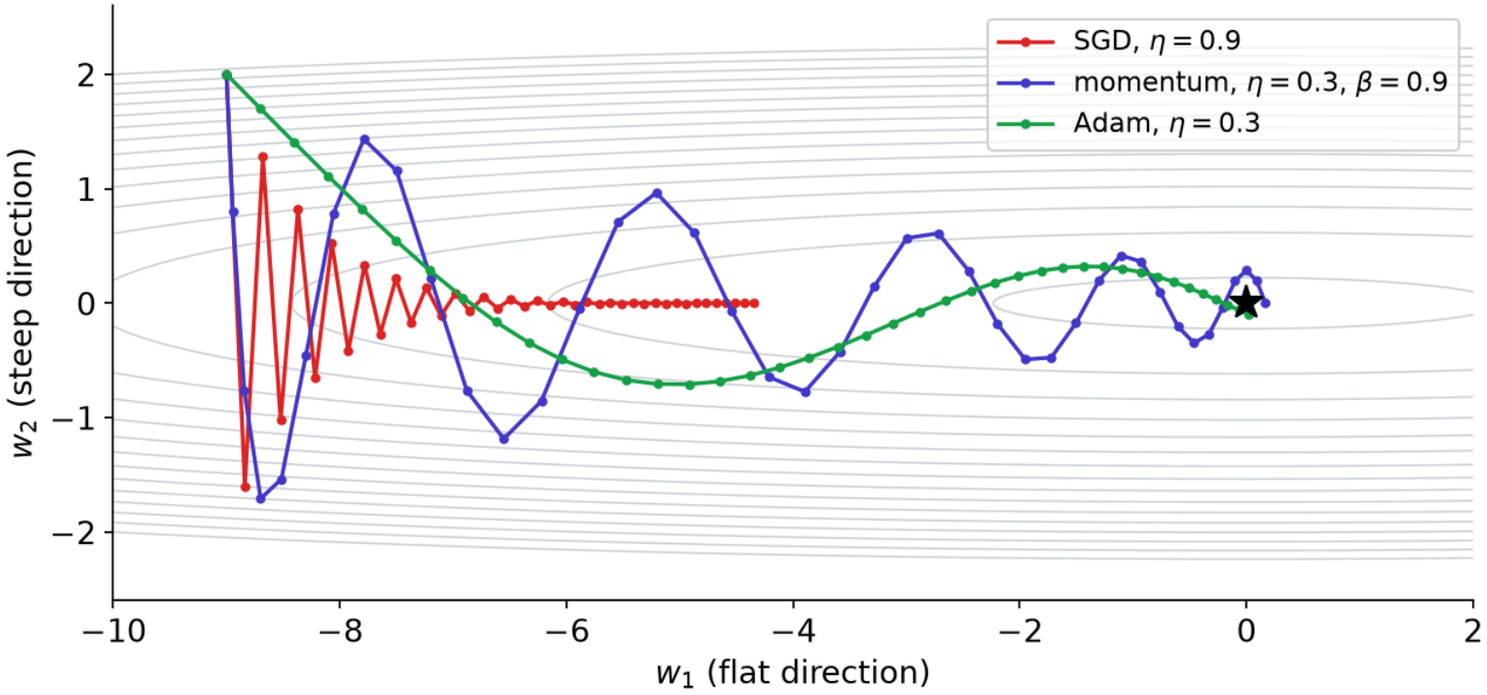
Adam: momentum plus a step size per parameter

$$m \leftarrow \beta_1 m + (1-\beta_1)\, g, \qquad s \leftarrow \beta_2 s + (1-\beta_2)\, g^2 \qquad \text{(element-wise)}$$

$$\hat{m} = \frac{m}{1-\beta_1^t}, \qquad \hat{s} = \frac{s}{1-\beta_2^t}, \qquad w \leftarrow w - \eta\, \frac{\hat{m}}{\sqrt{\hat{s}} + \varepsilon}$$

- m: average gradient (momentum). s: average squared gradient, the typical size of g for this parameter.
- Dividing by $\sqrt{\hat{s}}$: parameters with small gradients get bigger steps, parameters with large gradients smaller ones.
- **Bias correction**: m and s start at 0, so after step 1, $m = (1-\beta_1)g = 0.1\, g$, far too small. Dividing by $1-\beta_1^t$ fixes it: $\hat{m} = g$ at $t = 1$. The correction vanishes when t grows.
- Defaults: $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$ (Kingma and Ba, 2015).

Three optimisers on the same narrow valley



Cost $J = w_1^2/100 + w_2^2$, start $(-9, 2)$, 40 steps each. SGD zigzags then crawls (it stops at $w_1 = -4.35$); momentum and Adam reach the minimum.

Exercise 4 · Two steps of SGD, momentum and Adam

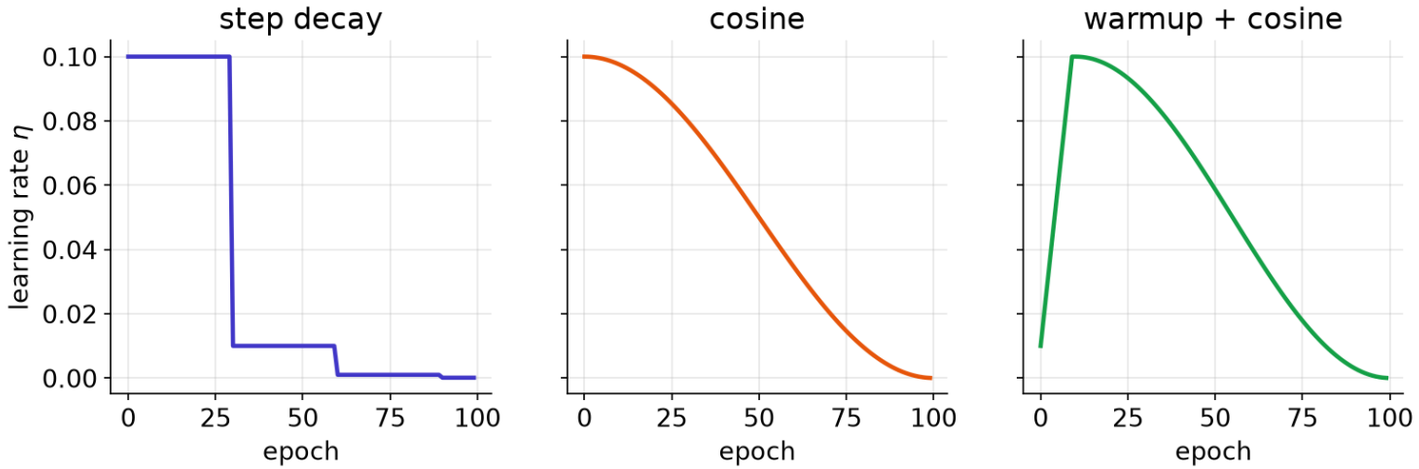
Minimise $J(w) = w^2$ (so $g = 2w$) from $w = 1$, with $\eta = 0.1$.

Exercise 7 min pen and paper

1. Plain gradient descent: compute w after 1 and 2 steps.
2. Momentum with $\beta = 0.9$, $v = 0$ at the start: compute g, v, w after 1 and 2 steps. Which method is closer to the minimum?
3. Adam, first step only ($m = s = 0$, neglect ε): show that $\hat{m} = g$ and $\hat{s} = g^2$. What is the size of the first step, whatever g?

Your notes / answers (saved in this browser)

Learning-rate schedules



- **Large steps first** (move fast), **small steps at the end** (settle in a minimum).
- Step decay: divide η by 10 every few epochs. Cosine: smooth decrease to 0. Warmup: increase η linearly during the first steps (Adam's statistics are unreliable at the start; standard for transformers).

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=n_epochs)
for epoch in range(n_epochs):
    train_one_epoch(model, train_loader, loss_fn, optimizer)
    scheduler.step()                                    # once per epoch, after the optimiser steps
```

Diagnosing a training run

Symptom	Likely cause	Fix
Loss becomes huge, then `nan`	Learning rate too large, exploding gradients	Divide η by 10; gradient clipping; check the inputs are normalised
Loss flat from the start (at $\ln C$)	η too small; a bug (no `step()`, no `zero_grad()`, labels shuffled); dead ReLUs; deep sigmoid network	Overfit one small batch first; check the loop; ReLU + He init, batch norm
Train loss goes down, validation goes up	Overfitting	More data, weight decay, dropout, early stopping
Both losses stay high and close	Underfitting	Bigger model, train longer, larger η, less regularisation
Loss very noisy, does not settle	η too large for the end of training, batch too small	Schedule (decay), larger batch

With C classes, a network that knows nothing predicts $1/C$ everywhere: loss $= -\ln(1/C) = \ln C$, that is $\ln 10 = 2.30$ for FashionMNIST.

Gradient clipping

If the norm of the whole gradient vector g (all parameters together) exceeds a threshold c, rescale it:

$$g \leftarrow \begin{cases} g & \text{if } \|g\| \leq c \\ c\,\dfrac{g}{\|g\|} & \text{otherwise} \end{cases}$$

The direction is kept, only the length is limited. Example: $g = (3, 4)$, $\|g\| = 5$, $c = 1$ gives $g = (0.6, 0.8)$.

```python
from torch.nn.utils import clip_grad_norm_

optimizer.zero_grad()
loss = loss_fn(model(xb), yb)
loss.backward()
clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

Between `backward()` and `step()`. Standard for recurrent networks (session 8), a cheap safety net elsewhere.

Exercise 5 · Four runs to diagnose



Exercise 5 min think

For each run (A, B, C, D): what is happening, and what is the first thing you change? Why is the flat curve of run C at 2.30 exactly?

Your notes / answers (saved in this browser)

Break: 10 minutes

TP · Train a better network on FashionMNIST

TP 80 min tp/TP06_better_training.ipynb

1. Part A: provoke overfitting (big network, 2 000 images) and measure the gap (10 min).
2. Part B: weight decay, dropout, early stopping, written by you; a comparison table (25 min).
3. Part C: batch norm by hand, checked against `nn.BatchNorm1d` (15 min).
4. Part D: momentum and Adam by hand, then SGD versus momentum versus Adam, a cosine schedule (15 min).
5. Part E: the training doctor, three broken runs to diagnose and fix (15 min).

Deliverable: every cell prints "Exercise validated!" and the comparison table of Part B is filled.

Your notes / answers (saved in this browser)

Watch it on YouTube Machine Learnia, Deep Learning series

Review the foundations of this course.

Video 10

Réseau de neurones à N couches

A neural network with N layers

Playlist

The whole Deep Learning series

10 videos, from the perceptron to the network with N layers

Videos in French. Click a line in your copy of the course to open the video.

Summary

1. Overfitting is read on the curves: train loss down, validation loss up. Always plot both.
2. Weight decay shrinks weights ($w \leftarrow (1 - \eta\lambda)w - \eta g$); dropout drops neurons and rescales by $1/(1-p)$; early stopping keeps the best validation checkpoint.
3. Batch norm normalises each neuron over the batch (running statistics at test time); layer norm normalises each example over its features.
4. Momentum accumulates gradients; Adam adds a per-parameter step size and bias correction; schedules decrease η.
5. Diagnose from curves: `nan` means η too large, flat at $\ln C$ means nothing is learned.

Project: add train and validation monitoring curves and a failure diagnosis section to your repository (tag `m2`).

Next session: images have a structure. Convolutional neural networks.